VEÍCULO AUTÔNOMO EM MINIATURA: INTELIGÊNCIA ARTIFICIAL APLICADA À SIMULAÇÃO DE TRÂNSITO URBANO

Emilly da Silva Machado Souza[1]; Rynalde Gabriel Silva Serejo[2]; Pedro Hiago Ferreira da Silva[3]; Ana Caroline Meireles Soares[4]

Resumo

Veículos autônomos em escala reduzida são plataformas de pesquisa acessíveis, mas o processador embarcado limita o uso de inteligência artificial. O objetivo deste trabalho foi desenvolver e validar modelos de inteligência artificial para um carro autônomo em miniatura, com direção *Ackermann*, *Raspberry Pi* 4 e *webcam* Logitech C270, que siga a faixa e obedeça à sinalização do regulamento de 2026 da competição FIRA. Depois da construção do protótipo físico, o desenvolvimento passou para a simulação, no *Gazebo* com ROS 2. Foram gerados circuitos e uma cidade em miniatura, imagens rotuladas automaticamente, com randomização de domínio, e duas redes convolucionais compactas, quantizadas em INT8: uma para faixas (0,61 milhão de parâmetros) e outra para placas com pictogramas (0,16 milhão). Em seguida, construiu-se um gêmeo digital, que reproduz o chassi modelado e impresso em 3D, a direção por servomotor e a câmera real. Em oito cenários adversos, a rede de faixas teve menor erro lateral em sete e completou cinco percursos, contra dois da abordagem clássica, e o classificador reconheceu 98,6% de 1.616 placas. No gêmeo, com os 43° de campo de visão da câmera real, os modelos anteriores não atravessaram nenhum *checkpoint*. Com a percepção adaptada e a rede retreinada com 13.749 imagens dessa câmera, o veículo completou os circuitos e os percursos urbanos sem penalidades. Conclui-se que a simulação permite desenvolver modelos que rodam em CPU embarcada e que, para levá-los ao robô, a câmera simulada precisa reproduzir a câmera real.

[1] Estudante do Curso de Bacharelado em Engenharia da Computação – IFMA Campus Santa Inês. E-mail: emy.engtech@gmail.com
[2] Estudante do Curso de Bacharelado em Engenharia da Computação – IFMA Campus Santa Inês. E-mail: rynaldegabrielsilvaserejo@gmail.com
[3] Estudante do Curso de Bacharelado em Engenharia da Computação – IFMA Campus Santa Inês. E-mail: pedro.hiago@acad.ifma.edu.br
[4] Orientadora. Docente do IFMA Campus Santa Inês. E-mail: ana.soares@ifma.edu.br

Palavras-chave: *Veículo autônomo. Simulação robótica. Redes neurais convolucionais. Gêmeo digital. Sistemas embarcados.*

Financiamento: IFMA/PRPGI, Programa Institucional de Bolsas de Iniciação em Desenvolvimento Tecnológico e Inovação (PIBITI), Edital nº 09/2025.

Introdução

Com o aprendizado profundo, os sistemas de condução autônoma passaram a interpretar imagens de câmeras para seguir a faixa e reconhecer a sinalização em tempo real. Alguns separam percepção, planejamento e controle em módulos; outros convertem a imagem diretamente em comandos, numa abordagem de ponta a ponta (Chen *et al.*, 2024; Chib; Singh, 2024). Um carro em escala reduzida enfrenta os mesmos problemas de percepção, controle e integração de um veículo real, com custo muito menor.

A competição de carros autônomos da *Federation of International RoboSports Association* (FIRA) oferece um cenário padronizado para esse estudo. O regulamento de 2026 exige um veículo elétrico em escala 1:10, com direção *Ackermann*, que percorra um circuito delimitado por faixas e uma cidade em miniatura com faixas de pedestres, linhas de parada e placas. A comunicação sem fio é proibida durante as provas, e o processamento embarcado tem coeficiente de pontuação duas vezes maior que o externo (FIRA, 2025). Percepção e controle precisam, assim, rodar no computador de bordo. Em projetos de baixo custo, esse computador costuma ser uma placa como o *Raspberry Pi* 4, que não tem acelerador para redes neurais e dificilmente executa detectores de uso geral, como os da família YOLO, comuns no reconhecimento de placas (Luo *et al.*, 2024; Terven; Córdova-Esparza; Romero-González, 2023). Por isso, a primeira etapa deste projeto usou navegação por visão computacional clássica, com a biblioteca *OpenCV* (Bradski, 2000).

A simulação permite desenvolver inteligência artificial sem depender só do protótipo físico. O *Gazebo* simula sensores, dinâmica e cenários (Koenig; Howard, 2004), e, com o *Robot Operating System* 2 (ROS 2), os mesmos programas rodam no simulador e no robô (Macenski *et al.*, 2022). Como o simulador conhece a posição de cada marcação, as imagens podem ser rotuladas automaticamente, e a randomização de domínio, que varia cores, iluminação e geometria durante a geração dos dados, ajuda os modelos a generalizar (Tobin *et al.*, 2017). A dificuldade está na transferência para o robô, por causa da diferença entre simulação e realidade (Zhao; Queralta; Westerlund,

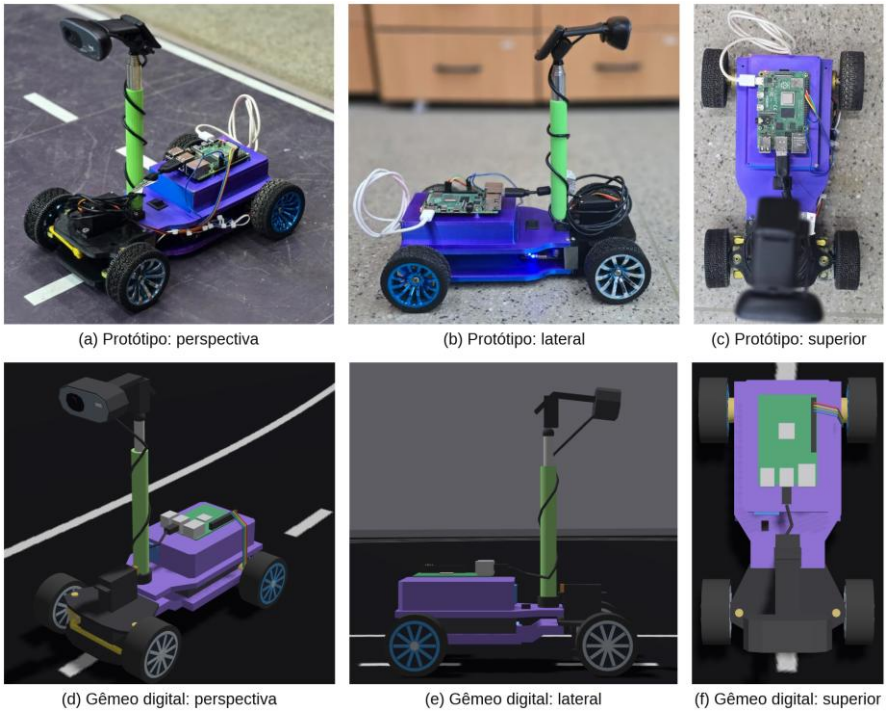
2020). Essa diferença diminui quando o modelo virtual reproduz com fidelidade o sistema físico, que é a proposta do gêmeo digital (Grieves; Vickers, 2017).

O objetivo deste trabalho foi desenvolver e testar em simulação modelos de inteligência artificial para detectar faixas e reconhecer placas no protótipo físico do projeto, dentro das restrições de um processador embarcado, compará-los com a abordagem clássica e verificá-los em um gêmeo digital do protótipo, com a câmera real.

Metodologia

A pesquisa é aplicada e experimental, de desenvolvimento tecnológico, e foi realizada no Instituto Federal de Educação, Ciência e Tecnologia do Maranhão (IFMA), Campus Santa Inês, entre setembro de 2025 e agosto de 2026, sem envolver seres humanos ou animais. Na primeira etapa, a equipe construiu o protótipo físico (Figura 1a-c), com chassi impresso em 3D, direção *Ackermann* acionada por servomotor MG995, tração traseira por motores de 12 V com redução, *Raspberry Pi* 4 (4 GB) e *webcam* Logitech C270 em um mastro. A navegação clássica do protótipo foi testada no laboratório do campus e na etapa regional da FIRA Brasil (Bacabal, MA, março de 2026).

Figura 1 — Protótipo físico e seu gêmeo digital no *Gazebo*, abaixo, nas mesmas vistas



Fonte: Elaborada pelos autores (2026).

Na segunda etapa, os cenários da competição foram reproduzidos no *Gazebo Harmonic* com o ROS 2 *Jazzy*. Um programa gera os cenários e confere cada um com o

regulamento. O circuito de corrida tem 21,41 m de volta e largura entre 46,3 e 53,7 cm. A cidade tem 3 × 3 cruzamentos, faixas de pedestres, linhas de parada e placas *AprilTag* 36h11 (Olson, 2011) ou pictogramas. Todos os cenários existem nas duas colorações previstas, com marcações brancas ou pretas. A simulação usou primeiro um veículo genérico, com câmera de 92° de campo de visão a 15 cm de altura. No período final, passou a usar um gêmeo digital do protótipo (Figura 1d-f), que reproduz o robô final, com chassi modelado no *Autodesk Fusion* e impresso em 3D, entre-eixos de 200 mm, direção por servomotor e motores traseiros de corrente contínua acionados por PWM. A câmera simula a C270 a 640 × 480 pixels, com distância focal de 816 pixels, média de duas calibrações publicadas (FIRST Tech Challenge, 2023; IRI, 2018); isso equivale a 42,8° de campo de visão. Ela fica na mesma posição e com a mesma inclinação que no robô, e uma cópia da pista do laboratório renderizada no *Gazebo* com essa pose coincide com um quadro real capturado pelo protótipo (Figura 2a-b).

O software é modular. A percepção clássica e a rede neural publicam a mesma mensagem, com deslocamento lateral, orientação e curvatura da faixa, e são intercambiáveis. A percepção clássica projeta a imagem em vista de cima por homografia (Hartley; Zisserman, 2004) e binariza as marcações pelo método de Otsu (1979). Um controlador proporcional-derivativo com antecipação da curvatura (Åström; Murray, 2008) conduz o veículo, e um supervisor implementado como máquina de estados para o carro por 3,4 s em cada cruzamento, lê a placa e faz a conversão. Como a C270 não enxerga os primeiros 0,81 m à frente do carro e perde a lateral interna nas curvas, duas mudanças foram feitas para ela. O detector passou a classificar cada marcação visível (lateral esquerda, central ou lateral direita) com base no quadro anterior, de modo que uma única marcação já indica o centro da faixa. O controlador passou a mirar um ponto 1 m adiante, por perseguição pura (Coulter, 1992).

Os dados foram gerados sem anotação manual: a posição das marcações na imagem é calculada por projeção, no formato de âncoras de linha (Qin; Wang; Li, 2020), com 32 linhas, três marcações e 64 células. Um condutor automático percorreu 14 circuitos randomizados (cores, iluminação, largura da pista e das marcações, traços e duas geometrias), oscilando de propósito dentro da faixa, e gerou 11.200 imagens. Para a C270, o gêmeo percorreu os mesmos circuitos, e 12 cidades randomizadas completaram o conjunto, num total de 13.749 imagens. Mundos inteiros ficaram reservados para validação, e nenhum cenário de teste entrou no treinamento. A rede de faixas,

TangerinaLaneNet, tem 609.982 parâmetros e usa apenas convoluções 3 × 3 com normalização em lote e ativação ReLU6 (Sandler *et al.*, 2018), que se quantizam bem em inteiros de 8 bits (Jacob *et al.*, 2018). A entrada é a imagem em tons de cinza, com 256 × 128 pixels. A rede foi treinada em *PyTorch* (Paszke *et al.*, 2019) com o otimizador AdamW (Loshchilov; Hutter, 2019) e quantizada em INT8 para o *ONNX Runtime* (ONNX Runtime Developers, 2018). O arquivo do modelo guarda a câmera usada no treinamento, e o nó de inferência recusa uma lente cuja distância focal difira em mais de 2%. O classificador de placas, *TangerinaSignNet*, aproveita a altura das placas, igual à da câmera: uma rede totalmente convolucional de 158.547 parâmetros percorre só uma faixa horizontal de 384 × 72 pixels e indica a classe, o tamanho e a posição de cada placa. Ela foi treinada com 4.978 quadros e 13.405 placas, em 12 mundos.

As avaliações usaram a verdade de campo do simulador. As medidas foram a raiz do erro quadrático médio (RMS) do deslocamento lateral e a pontuação oficial (FIRA, 2025). Na corrida, $S_{\mathrm{AR}} = (1 + \max\{(T_{\mathrm{stage}} - T_{\mathrm{total}})/T_{\mathrm{stage}}, 0\}) \times 35 \times cp$, em que T_{stage} é o tempo da prova, T_{total} é o tempo gasto e cp é o número de *checkpoints* atravessados. No percurso urbano, a pontuação tem a mesma forma, multiplicada pelo coeficiente de reconhecimento K_{s}. Com o veículo genérico, as duas percepções foram comparadas em oito mundos adversos, e o percurso urbano foi executado repetidas vezes. Com o gêmeo, foram executados os dois circuitos e os dois percursos urbanos de referência. Os tempos de inferência foram medidos em CPU, com dois a quatro *threads*.

Resultados e Discussão

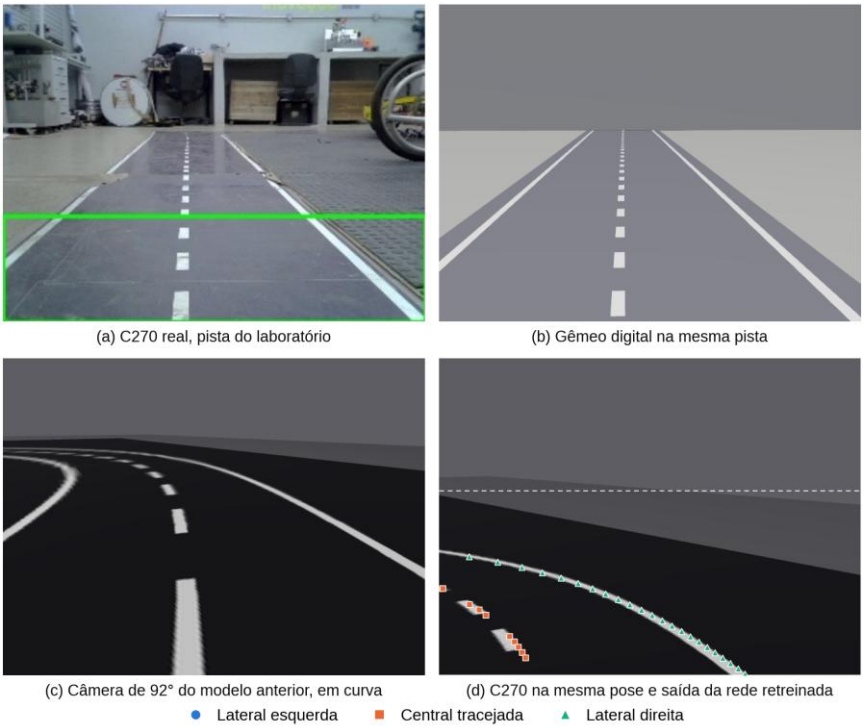
Nos mundos de validação, a *TangerinaLaneNet* do veículo genérico errou em média 0,95 pixel na posição das marcações e acertou a presença em 92,2% dos casos; a versão retreinada para a C270 errou 1,84 pixel e acertou 93,8%. Em INT8, as redes processam um quadro em menos de 1 ms na CPU de desenvolvimento, bem abaixo do orçamento de 67 ms (15 quadros por segundo) do processador embarcado. Nas condições adversas, a rede se saiu melhor. Em oito mundos (iluminação fraca, ofuscamento, marcações de 1 cm, traços longos, largura variando ±10%, largada com luminância próxima à das marcações, ruído de câmera e obstáculos), ela teve menor erro lateral em sete, por exemplo 2,66 contra 5,70 cm com marcações de 1 cm, e completou cinco percursos, contra dois da abordagem clássica. Isso é coerente com a randomização de domínio: como cores e iluminação variam no treinamento, o modelo passa a depender da forma das marcações (Tobin *et al.*, 2017). Nenhuma das abordagens desvia de obstáculos.

Com a faixa encoberta, a rede errou com confiança (15,6 cm), enquanto a percepção clássica declarou inválidos dois terços dos quadros; por isso, convém manter as duas em paralelo.

O classificador de placas reconheceu 98,58% das 1.616 placas dos quatro mundos de validação, com 0,010 falso positivo por quadro e 0,46 ms por quadro em INT8; as únicas confusões foram entre as setas espelhadas, siga à esquerda e siga à direita. Com o veículo genérico, percepção de faixas clássica e leitura de placas, o carro completou o percurso urbano nas quatro variantes (duas colorações, com *AprilTags* ou pictogramas), sem penalidades. Com pictogramas, o coeficiente $K_s = 1{,}3$ elevou a pontuação para cerca de 313 pontos; com *AprilTags*, ela ficou em 221 e 241. Em execuções repetidas, o carro completou o percurso em 30 de 33 tentativas, e as três falhas foram derivas laterais depois de uma conversão, sem relação com a leitura das placas.

Guiado pela posição real do simulador, o gêmeo digital completou os dois circuitos com erro lateral máximo de 2,5 cm; o modelo de direção e tração, portanto, atende às provas. Já a câmera exigiu mudanças (Figura 2c-d). Com 42,8° de campo de visão, a C270 enxerga as duas laterais da pista em apenas 26% a 29% das poses de uma volta, e a câmera de 92°, em todas; inclinar a C270 não altera essa proporção. Sem alterações, as duas percepções falharam, e o carro não atravessou nenhum *checkpoint* (Tabela 1). Com as mudanças descritas, a percepção clássica completou os quatro cenários. A rede retreinada só completou os circuitos depois que o nó passou a estimar o centro da faixa a partir de uma marcação isolada, e só completou a cidade depois que as 12 cidades entraram no treinamento. Após a conversão à direita, a C270 vê apenas faixa de pedestres, linha de parada e esquina, cena que nenhum circuito mostra. Na cidade de marcações brancas, a rede ainda saiu dessa conversão 26 cm à direita do centro da rua e atravessou um *checkpoint* fora da faixa.

Figura 2 — Câmera do gêmeo digital. Acima, o quadro real da C270 e o quadro do gêmeo digital na mesma pista; abaixo, as câmeras do veículo genérico e da C270 na mesma pose, na entrada de uma curva, com as marcações previstas pela rede retreinada abaixo da linha tracejada



Fonte: Elaborada pelos autores (2026). Em (a), o retângulo verde é a região de interesse do programa do protótipo.

Tabela 1 — Gêmeo digital com a C270 nos cenários de referência: *checkpoints* e pontuação

Percepção	**Corrida, brancas**	**Corrida, pretas**	**Cidade, brancas**	**Cidade, pretas**
Clássica original	0/8	0/8	2/4, parou	1/4, saiu da rua
Rede para 92°	0/8	0/8	não testada	não testada
Clássica adaptada	8/8; 438,7	8/8; 437,9	4/4; 240,1	4/4; 241,6
Rede retreinada	8/8; 439,9	8/8; 439,9	3/4; 181,2	4/4; 241,6

Fonte: Elaborada pelos autores (2026).

Nota: pontuação S_{AR} na corrida e S_{AUD} na cidade; uma execução por cenário; na cidade, contam-se os *checkpoints* atravessados na faixa correta, com placas lidas por *AprilTag*.

Nesses testes, a maior diferença entre simulação e realidade veio da geometria da câmera (Zhao; Queralta; Westerlund, 2020). A rede treinada com outra lente falhou por completo, e a correção exigiu dados vistos pela câmera real e uma percepção adequada a um campo de visão estreito. No robô, essa falha apareceria sem diagnóstico; como o modelo guarda a câmera do treinamento, o nó de inferência recusa outra lente já na inicialização. As redes ficaram muito abaixo do limite de cerca de 5 milhões de parâmetros graças à formulação por âncoras de linha, mais leve que a segmentação da imagem (Qin; Wang; Li, 2020), e às operações que se quantizam bem (Jacob *et al.*, 2018); detectores de uso geral exigiriam aceleração dedicada (Terven; Córdova-Esparza; Romero-González, 2023). Como limitações, os testes ocorreram em simulação e em

processador x86, a pose da câmera ainda precisa ser conferida no robô, cada cenário do gêmeo foi executado uma vez, e o classificador de pictogramas ainda não foi retreinado para a C270.

Conclusões

A simulação no *Gazebo* com ROS 2 permite treinar e testar modelos de inteligência artificial para o veículo em miniatura sem anotação manual, com a pontuação oficial como medida. As redes cabem em uma CPU embarcada, erram menos que a abordagem clássica quando a imagem se degrada e obtêm a bonificação por pictogramas. No gêmeo digital, modelos treinados para outra lente falham por completo, enquanto a percepção adaptada e retreinada para a C270 completa os circuitos e os percursos urbanos em simulação. Para transferir os modelos, a câmera simulada precisa reproduzir a câmera do robô. Ainda faltam testes no *hardware*, onde o desempenho no *Raspberry Pi* 4 precisa ser medido, o desvio de obstáculos e a medida da confiabilidade do gêmeo. Os próximos passos são embarcar os modelos, conferir no robô a pose da câmera e as dimensões do carro, retreinar o classificador de placas para a C270 e implementar o desvio de obstáculos.

Agradecimentos

Ao IFMA Campus Santa Inês e à PRPGI, pela bolsa PIBITI e pela infraestrutura do Laboratório de Eletromecânica, e à organização da FIRA Brasil, que permitiu testar o protótipo em competição.

Referências

ÅSTRÖM, K. J.; MURRAY, R. M. **Feedback systems**: an introduction for scientists and engineers. Princeton: Princeton University Press, 2008. DOI: 10.1515/9781400828739.

BRADSKI, G. The OpenCV library. **Dr. Dobb's Journal of Software Tools**, [*s. l.*], v. 25, n. 11, p. 120-125, 2000.

CHEN, L. *et al.* End-to-end autonomous driving: challenges and frontiers. **IEEE Transactions on Pattern Analysis and Machine Intelligence**, [*s. l.*], v. 46, n. 12, p. 10164-10183, 2024. DOI: 10.1109/TPAMI.2024.3435937.

CHIB, P. S.; SINGH, P. Recent advancements in end-to-end autonomous driving using deep learning: a survey. **IEEE Transactions on Intelligent Vehicles**, [*s. l.*], v. 9, n. 1, p. 103-118, 2024. DOI: 10.1109/TIV.2023.3318070.

COULTER, R. C. **Implementation of the pure pursuit path tracking algorithm**. Pittsburgh: Carnegie Mellon University, Robotics Institute, 1992. (Technical Report CMU-RI-TR-92-01).

FIRA. **FIRA Challenge – Autonomous Cars**: laws of the game (Pro) 2026. Versão 2.0.1. [*S. l.*], 2025.

FIRST TECH CHALLENGE. **Webcams for FTC VisionPortal**. [*S. l.*], 2023. Disponível em: https://github.com/FIRST-Tech-Challenge/FtcRobotController/wiki/Webcams-for-FTC-VisionPortal. Acesso em: 29 set. 2026.

GRIEVES, M.; VICKERS, J. Digital twin: mitigating unpredictable, undesirable emergent behavior in complex systems. *In*: KAHLEN, F.-J.; FLUMERFELT, S.; ALVES, A. (ed.). **Transdisciplinary perspectives on complex systems**: new findings and approaches. Cham: Springer, 2017. p. 85-113. DOI: 10.1007/978-3-319-38756-7_4.

HARTLEY, R.; ZISSERMAN, A. **Multiple view geometry in computer vision**. 2. ed. Cambridge: Cambridge University Press, 2004. DOI: 10.1017/CBO9780511811685.

IRI – INSTITUT DE ROBÒTICA I INFORMÀTICA INDUSTRIAL. **iri_uvc_camera_bringup**: calibration file for the Logitech C270 camera at 640x480. Barcelona, 2018. Disponível em: https://gitlab.iri.upc.edu/labrobotica/ros/sensors/mono_cameras/iri_uvc_camera_bringup. Acesso em: 29 set. 2026.

JACOB, B. *et al.* Quantization and training of neural networks for efficient integer-arithmetic-only inference. *In*: IEEE/CVF CONFERENCE ON COMPUTER VISION AND PATTERN RECOGNITION, 2018, Salt Lake City. **Proceedings** [...]. Piscataway: IEEE, 2018. p. 2704-2713. DOI: 10.1109/CVPR.2018.00286.

KOENIG, N.; HOWARD, A. Design and use paradigms for Gazebo, an open-source multi-robot simulator. *In*: IEEE/RSJ INTERNATIONAL CONFERENCE ON INTELLIGENT ROBOTS AND SYSTEMS, 2004, Sendai. **Proceedings** [...]. Piscataway: IEEE, 2004. v. 3, p. 2149-2154. DOI: 10.1109/IROS.2004.1389727.

LOSHCHILOV, I.; HUTTER, F. Decoupled weight decay regularization. *In*: INTERNATIONAL CONFERENCE ON LEARNING REPRESENTATIONS, 7., 2019, New Orleans. **Proceedings** [...]. [*S. l.*]: ICLR, 2019. Disponível em: https://arxiv.org/abs/1711.05101. Acesso em: 29 set. 2026.

LUO, Y. *et al.* A YOLOv8-CE-based real-time traffic sign detection and identification method for autonomous vehicles. **Digital Transportation and Safety**, [*s. l.*], v. 3, n. 3, p. 82-91, 2024. DOI: 10.48130/dts-0024-0009.

MACENSKI, S. *et al.* Robot Operating System 2: design, architecture, and uses in the wild. **Science Robotics**, [*s. l.*], v. 7, n. 66, eabm6074, 2022. DOI: 10.1126/scirobotics.abm6074.

OLSON, E. AprilTag: a robust and flexible visual fiducial system. *In*: IEEE INTERNATIONAL CONFERENCE ON ROBOTICS AND AUTOMATION, 2011,

Shanghai. **Proceedings** [...]. Piscataway: IEEE, 2011. p. 3400-3407. DOI: 10.1109/ICRA.2011.5979561.

ONNX RUNTIME DEVELOPERS. **ONNX Runtime**. [*S. l.*]: Microsoft, 2018. Disponível em: https://onnxruntime.ai. Acesso em: 29 set. 2026.

OTSU, N. A threshold selection method from gray-level histograms. **IEEE Transactions on Systems, Man, and Cybernetics**, [*s. l.*], v. 9, n. 1, p. 62-66, 1979. DOI: 10.1109/TSMC.1979.4310076.

PASZKE, A. *et al.* PyTorch: an imperative style, high-performance deep learning library. *In*: CONFERENCE ON NEURAL INFORMATION PROCESSING SYSTEMS, 33., 2019, Vancouver. **Advances in Neural Information Processing Systems 32**. Red Hook: Curran Associates, 2019. p. 8024-8035. Disponível em: https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html. Acesso em: 29 set. 2026.

QIN, Z.; WANG, H.; LI, X. Ultra fast structure-aware deep lane detection. *In*: EUROPEAN CONFERENCE ON COMPUTER VISION, 16., 2020, Glasgow. **Computer Vision – ECCV 2020**: proceedings, part XXIV. Cham: Springer, 2020. p. 276-291. DOI: 10.1007/978-3-030-58586-0_17.

SANDLER, M. *et al.* MobileNetV2: inverted residuals and linear bottlenecks. *In*: IEEE/CVF CONFERENCE ON COMPUTER VISION AND PATTERN RECOGNITION, 2018, Salt Lake City. **Proceedings** [...]. Piscataway: IEEE, 2018. p. 4510-4520. DOI: 10.1109/CVPR.2018.00474.

TERVEN, J.; CÓRDOVA-ESPARZA, D.-M.; ROMERO-GONZÁLEZ, J.-A. A comprehensive review of YOLO architectures in computer vision: from YOLOv1 to YOLOv8 and YOLO-NAS. **Machine Learning and Knowledge Extraction**, [*s. l.*], v. 5, n. 4, p. 1680-1716, 2023. DOI: 10.3390/make5040083.

TOBIN, J. *et al.* Domain randomization for transferring deep neural networks from simulation to the real world. *In*: IEEE/RSJ INTERNATIONAL CONFERENCE ON INTELLIGENT ROBOTS AND SYSTEMS, 2017, Vancouver. **Proceedings** [...]. Piscataway: IEEE, 2017. p. 23-30. DOI: 10.1109/IROS.2017.8202133.

ZHAO, W.; QUERALTA, J. P.; WESTERLUND, T. Sim-to-real transfer in deep reinforcement learning for robotics: a survey. *In*: IEEE SYMPOSIUM SERIES ON COMPUTATIONAL INTELLIGENCE, 2020, Canberra. **Proceedings** [...]. Piscataway: IEEE, 2020. p. 737-744. DOI: 10.1109/SSCI47803.2020.9308468.